

EJERCICIO 1

Lo que se busca son las líneas que parecen títulos: todo en mayúsculas, con al menos dos palabras y que no terminen con punto.

Regex:

```
^[A-ZÁÉÍÓÚÜÑ][A-ZÁÉÍÓÚÜÑ0-9\s\-\-]*\s[A-ZÁÉÍÓÚÜÑ0-9][A-ZÁÉÍÓÚÜÑ0-9\s\-\-]*(?<!\. )$
```

Explicación:

El ^ indica que el patrón empieza desde el inicio de la línea y junto con re.MULTILINE esto hace que el ^ y el \$ se evalúen línea por línea en lugar de aplicarse al texto completo.

La clase [A-ZÁÉÍÓÚÜÑ] al inicio le pide que el primer carácter sea una letra mayúscula del español, con acentos y Ñ incluidos.

El \s en medio del patrón obliga a que haya al menos un espacio, lo que garantiza que haya mínimo dos palabras, sin ese espacio una sola palabra en mayúsculas podría entrar.

Al final el (?<!\.)\$ es un lookbehind negativo, básicamente revisa que el carácter justo antes del fin de línea no sea un punto, el \$ marca el final y el (?<!\.) dice que no puede haber un punto justo antes.

EJERCICIO 2

Se detectan fechas en dos formatos: DD/MM/YYYY y YYYY-MM-DD. La regex busca los dos a la vez usando alternancia.

Regex:

```
\b(\d{2}/\d{2}/\d{4}|\d{4}-\d{2}-\d{2})\b
```

Explicación:

El \b al inicio y al final es el límite de palabra, para que la fecha no se capture si está pegada a otras letras o números sin separación.

El \d{2} busca exactamente dos dígitos para el día y el mes y \d{4} busca cuatro para el año.

La barra / y el guión - son caracteres literales, el patrón los busca tal cual en el texto.

El | en medio es la alternancia y significa 'o esto o lo otro', entonces busca el formato con diagonal o el formato con guiones.

EJERCICIO 3

Atrapa correos con el patrón clásica parte local, arroba, dominio y extensión, se excluyen automáticamente los que no tienen extensión porque el patrón la exige.

Regex:

```
[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}
```

Explicación:

La parte `[a-zA-Z0-9._%+\-]+\` antes del `@` acepta letras, números y los símbolos típicos de un correo como punto, guión bajo o porcentaje, el `+` pide que haya al menos uno de esos.

La `@` es literal, como solo hay una en el patrón si el texto tiene `@@` no coincide bien.

Después viene `[a-zA-Z0-9.\-]+\` para el dominio que puede tener letras, números, puntos y guiones.

El `\.` es el punto escapado, sin el backslash el punto en regex significa cualquier carácter, entonces hay que escaparlo para que busque un punto de verdad.

Al final `[a-zA-Z]{2,}` es la extensión solo letras y mínimo dos caracteres por eso correos sin extensión como soporte@tecnm no entran.

EJERCICIO 4

Se quieren teléfonos que empiecen con 999, la LADA de Yucatán y estos pueden venir con paréntesis, guiones, espacios o sin nada.

Regex:

```
\(?:999\)?[\s\-\-]?[0-9]{3}[\s\-\-]?[0-9]{4}
```

Explicación:

El `\(?:` es el paréntesis de apertura escapado con el backslash, y el `?` lo hace opcional, lo mismo con el `\)?` de cierre entonces puede aparecer (999) o solo 999.

El 999 es literal y obligatorio si el número empieza diferente no coincide.

Los `[\s\-\-]?` son separadores opcionales, la clase acepta un espacio o un guión, y el `?` dice que puede haber uno o ninguno por eso funciona con 999-312-4455, con 999 312 4455 y con 9993124455.

El `[0-9]{3}` busca los siguientes tres dígitos y el `[0-9]{4}` los últimos cuatro, en total son 3 de la LADA más 3 más 4, que suman los 10 dígitos requeridos.

EJERCICIO 5

Se buscan cadenas independientes de entre 8 y 12 caracteres que tengan letras mayúsculas y números mezclados, tiene que haber al menos una letra y al menos un número.

Regex:

```
\b(?:[A-Z0-9]*[0-9])(?:[A-Z0-9]*[A-Z])[A-Z0-9]{8,12}\b
```

Explicación:

Los `\b` al inicio y al final son límites de palabra para que el identificador no se capture pegado a otros caracteres.

El `(?:[A-Z0-9]*[0-9])` es un lookahead positivo revisa hacia adelante sin consumir caracteres y verifica que en algún punto de la cadena haya al menos un dígito, si no hay ningún número falla aquí.

El `(?:[A-Z0-9]*[A-Z])` hace lo mismo pero para letras, verifica que haya al menos una mayúscula los dos juntos garantizan que la cadena sea verdaderamente alfanumérica.

La clase `[A-Z0-9]{8,12}` es la que hace la captura real, acepta letras mayúsculas y dígitos con mínimo 8 y máximo 12 caracteres.

EJERCICIO 6

Para cada línea del texto se obtienen todas las palabras de 3 o más letras y se revisa si alguna aparece más de una vez, si hay repetición esa línea va a los resultados.

Regex:

```
\b[a-zA-ZáéíóúüñÁÉÍÓÚÜÑ]{3,}\b
```

Explicación:

La regex se aplica con `findall()` a cada línea por separado para obtener la lista de palabras.

La clase `[a-zA-ZáéíóúüñÁÉÍÓÚÜÑ]` incluye letras del español con acentos y ñ, el `{3,}` pide mínimo 3 letras para no contar artículos cortos como 'de' o 'la' que se repiten naturalmente en casi cualquier oración.

`re.IGNORECASE` hace que 'Actividad' y 'actividad' se traten como la misma palabra.

Para detectar la repetición se recorre la lista de palabras y se va guardando cada una en otra lista llamada conteo, si al revisar una palabra ya estaba en conteo se marca la línea como repetida.

EJERCICIO 7

Se extraen los RFC y las CURP del texto. Los dos pueden aparecer junto a etiquetas como 'RFC:' o 'CURP:'

Regex:

RFC: `\b[A-ZÑ&]{3,4}\d{6}[A-Z0-9]{3}\b`

CURP: `b[A-Z]{4}\d{6}[HM][A-Z]{2}[A-Z]{3}[A-Z0-9]\d{1,2}\b`

Explicación:

La CURP empieza con 4 letras `[A-Z]{4}` de las iniciales de apellidos y nombre, luego los mismos 6 dígitos de fecha, el `[HM]` es el sexo registral, solo acepta H o M, lo que descarta automáticamente la CURP genérica del texto que tiene X ahí, después `[A-Z]{2}` es la clave del estado, `[A-Z]{3}` las consonantes internas y al final el dígito verificador.

EJERCICIO 8

Se buscan las líneas completas que contengan la palabra ERROR sin importar cómo esté escrita.

Regex:

`^.*ERROR.*$`

Explicación:

El `^` y el `$` marcan el inicio y el fin de línea, con `re.MULTILINE` se aplica línea por línea en lugar de al texto completo, que es lo que se necesita.

Los `.*` antes y después de ERROR capturan todo lo que haya en la misma línea el punto acepta cualquier carácter y el asterisco dice que puede haber ninguno o muchos, así se obtiene la línea completa, no solo la palabra.

`re.IGNORECASE` hace que la palabra coincida sin importar si está en mayúsculas, minúsculas o mezclada.